

Git - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

GIT — MASTER NOTES	5
A Complete Reference Book for Interviews, Production Work & System Understanding	5
Includes a full scenario-based command reference and real-world team collaboration workflows	5
TOPIC 1: REPOSITORY	5
1. Introduction	5
1.1 What is it?	5
1.2 Why was it introduced	5
2. Initializing and Cloning	5
3. The Three Areas — Working Directory, Staging Area, Repository	5
4. Real Life Analogy	6
5. Frequently Asked Interview Questions	6
TOPIC 2: BRANCHING	6
1. Introduction	6
2. Core Commands	6
3. How Branches Work Internally	7
4. Real Life Analogy	7
5. Frequently Asked Interview Questions	7
TOPIC 3: MERGE	7
1. Introduction	7
2. Two Types of Merges	7
Fast-Forward Merge (when main hasn't changed since the branch was created)	7
Three-Way (True) Merge (when main has ALSO changed since the branch was created)	8
3. Real Life Analogy	8
4. Edge Cases	8
5. Frequently Asked Interview Questions	8
TOPIC 4: REBASE	8
1. Introduction	8

2. Merge vs Rebase — The Key Visual Difference	9
3. Code Example	9
4. The Golden Rule of Rebasing — NEVER Rebase Shared/Public History	9
5. Interactive Rebase — Cleaning Up Commit History Before Sharing	9
6. Real Life Analogy	9
7. Frequently Asked Interview Questions	9

TOPIC 5: CHERRY-PICK **10**

1. Introduction	10
2. Code Example	10
3. Real Life Analogy	10
4. A Classic Real-World Use Case — Hotfixes	10
5. Edge Cases	10
6. Frequently Asked Interview Questions	11

TOPIC 6: STASH **11**

1. Introduction	11
2. Code Example	11
3. A Classic Real-World Scenario	11
4. Real Life Analogy	11
5. Frequently Asked Interview Questions	11

TOPIC 7: TAGS **12**

1. Introduction	12
2. Code Example	12
3. Lightweight vs Annotated Tags	12
4. Real Life Analogy	12
5. Frequently Asked Interview Questions	12

TOPIC 8: PULL REQUESTS **13**

1. Introduction	13
2. The Standard PR Workflow	13
3. Why PRs Matter for Team Collaboration (Critical for Real Jobs)	13

4. Real Life Analogy	13
5. Frequently Asked Interview Questions	14

TOPIC 9: CONFLICT RESOLUTION **14**

1. Introduction	14
2. Full Walkthrough — Resolving a Real Conflict	14
3. Real Life Analogy	15
4. Tools That Help	15
5. Edge Cases	15
6. Common Mistakes	15
7. Frequently Asked Interview Questions	15

TOPIC 10: REAL-WORLD TEAM WORKFLOWS & SCENARIO-BASED COMMAND REFERENCE **16**

1. Introduction	16
2. How Large Teams Avoid Conflicts on Shared Files — The Core Strategies	16
Strategy 1: Small, Short-Lived Feature Branches	16
Strategy 2: Frequent Integration (Pulling main Often)	16
Strategy 3: Branch Protection Rules (Enforced by GitHub/GitLab, Not Git Itself)	16
Strategy 4: Modular Code Architecture (Reduces WHO Touches the SAME Files)	16
Strategy 5: Trunk-Based Development vs Git Flow — Two Common Team Branching Models	17
Strategy 6: Feature Flags (Decoupling "merged" from "released")	17
Strategy 7: Code Owners (Automatically Requesting the RIGHT Reviewers)	17
3. Real Life Analogy	17
4. SCENARIO-BASED GIT COMMAND REFERENCE (Interview & Real-World Practice)	18
5. Common Mistakes	19
6. Best Practices	19
7. Frequently Asked Interview Questions	19
8. Revision Notes (Entire Git Section Consolidated)	20
9. Homework (Covers Entire Git Section)	20
FINAL SUMMARY — GIT COMPLETE	20

GIT — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

Includes a full scenario-based command reference and real-world team collaboration workflows

TOPIC 1: REPOSITORY

1. Introduction

1.1 What is it?

A Git repository ("repo") is a directory tracked by Git, storing the COMPLETE HISTORY of every change ever made to its files — Git is a **Distributed Version Control System (DVCS)**, meaning EVERY developer's local clone contains the FULL history, not just the current files (unlike older, centralized systems like SVN, where only the central server held the full history).

1.2 Why was it introduced

Before robust version control, teams tracked code changes via manually-named file copies (report_v2_final_FINAL.docx-style chaos) or centralized systems requiring constant network connectivity to a single server (a single point of failure, and slow for common operations like viewing history). Git (created by Linus Torvalds in 2005, for Linux kernel development) solves both: full history locally (fast, works offline), and a distributed model where every clone is a complete, independent backup.

2. Initializing and Cloning

```
git init # creates a NEW, empty repository in the current directory
git clone https://github.com/user/repo.git # copies an EXISTING remote repository, WITH its full history
```

3. The Three Areas — Working Directory, Staging Area, Repository

```
WORKING DIRECTORY STAGING AREA (INDEX) REPOSITORY (.git)
(your actual files, (changes marked to (permanent, committed
as you edit them) be included in the SNAPSHOTS - the actual
NEXT commit) history)
| | |
|-- git add file ----->| |
| |-- git commit ----->|
| |
|<----- git checkout / git reset -----|
```

```
git add file.txt # moves a change from Working Directory -> Staging Area
git commit -m "message" # moves staged changes from Staging Area -> Repository (permanent snapshot)
git status # shows the CURRENT state across all three areas
```

4. Real Life Analogy

- **Restaurant:** The Working Directory is like the ACTIVE prep counter (ingredients currently being chopped/mixed). The Staging Area is like a tray where you've DELIBERATELY set aside the SPECIFIC ingredients you've decided belong in THIS dish. The Repository is like the restaurant's PERMANENT, dated recipe archive — once a dish's exact recipe is filed there (committed), it's preserved forever, exactly as it was at that moment.

5. Frequently Asked Interview Questions

- 1 **What makes Git a "distributed" version control system?** Every clone contains the COMPLETE project history, not just the current files — unlike centralized systems where only a single server holds the full history.
- 2 **What are the three areas in Git's workflow?** Working Directory (your actual files), Staging Area/Index (changes marked for the next commit), Repository (the permanent, committed history).
- 3 **What's the difference between `git add` and `git commit`?** `git add` stages changes (Working Directory → Staging Area); `git commit` permanently records staged changes as a snapshot in the repository's history.

TOPIC 2: BRANCHING

1. Introduction

What is it? A branch is an INDEPENDENT line of development — a movable pointer to a specific commit, letting you work on new features/fixes WITHOUT affecting the main codebase until you're ready to merge that work back in.

2. Core Commands

```
git branch # list all LOCAL branches
git branch feature/login # create a new branch (doesn't switch to it yet)
git checkout feature/login # switch to that branch
git checkout -b feature/login # CREATE and SWITCH in one command (most common)
git switch feature/login # modern alternative to checkout, for SWITCHING branches specifically
git branch -d feature/login # delete a branch (safe - only if merged)
git branch -D feature/login # FORCE delete (even if unmerged - use carefully!)
```

3. How Branches Work Internally

Branches are just LIGHTWEIGHT POINTERS to a specific commit - NOT full copies of the codebase!

```
main: A---B---C
      |
      v
feature/login: D---E
```

'main' points to commit C. 'feature/login' points to commit E (which is built on top of C). Switching branches just MOVES which commit your working directory reflects - extremely fast, since Git doesn't need to duplicate the entire project.

4. Real Life Analogy

- **Restaurant:** A branch is like a chef experimenting with a NEW recipe VARIATION on a separate prep station, without disturbing the RESTAURANT'S currently-served, official recipe — if the experiment works out, it can later be adopted into the official menu (merged); if not, it's simply discarded, with zero impact on what's currently being served.

5. Frequently Asked Interview Questions

- 1 **What is a Git branch?** A movable pointer to a specific commit, representing an independent line of development.
- 2 **Why are branches considered "lightweight" in Git?** They're just pointers to a commit, not full copies of the project — creating/switching branches is fast and doesn't duplicate storage.
- 3 **What's the difference between `git branch -d` and `git branch -D`?** `-d` safely deletes a branch only if it's already been merged; `-D` force-deletes it regardless, even if unmerged work would be lost.

TOPIC 3: MERGE

1. Introduction

What is it? Merging COMBINES the changes from one branch INTO another — typically bringing a completed feature branch's work back into `main`.

2. Two Types of Merges

Fast-Forward Merge (when `main` hasn't changed since the branch was created)

```
git checkout main
git merge feature/login
# main simply MOVES FORWARD to point at feature/login's latest commit - no new merge commit needed
```

Before: main: A---B feature: A---B---C---D

After: main: A---B---C---D (main's pointer just moved forward)

Three-Way (True) Merge (when `main` has ALSO changed since the branch was created)

```
git checkout main
git merge feature/login
# Git creates a NEW "merge commit" with TWO parents, combining both histories

Before: main: A---B-----E
\
feature: C---D

After: main: A---B-----E---F (F is a new MERGE COMMIT, with parents E and D)
\ /
feature: C---D-/
```

3. Real Life Analogy

- **Restaurant:** A fast-forward merge is like the main menu simply ADOPTING a chef's finished experimental dish directly, since nothing else changed on the main menu in the meantime. A three-way merge is like COMBINING that chef's new dish AND some other unrelated menu updates that happened simultaneously — requiring a deliberate "here's how these two sets of changes come together" step.

4. Edge Cases

Case	Result
Both branches modified the SAME lines of the SAME file differently	A MERGE CONFLICT — Git cannot automatically decide which version to keep (full detail in Topic 9)
Merging a branch with NO new commits relative to <code>main</code>	Git reports "Already up to date," no changes occur

5. Frequently Asked Interview Questions

- 1 **What's the difference between a fast-forward merge and a three-way merge?** A fast-forward merge simply MOVES the target branch's pointer forward (no new commit, when the target hasn't diverged); a three-way merge creates a NEW merge commit with two parents, combining genuinely divergent histories.
- 2 **What causes a merge conflict?** Both branches modifying the SAME lines of the SAME file in DIFFERENT ways, leaving Git unable to automatically determine which change to keep.

TOPIC 4: REBASE

1. Introduction

What is it? Rebasing REWRITES your branch's commits so they appear to start from a NEWER base commit — producing a CLEAN, LINEAR history, as an alternative to merge's "combine with a merge commit" approach.

2. Merge vs Rebase — The Key Visual Difference

```
BEFORE (both scenarios start here):
main: A---B---E
\
feature: C---D

MERGE result (preserves the ACTUAL history, including the "branching" shape):
main: A---B---E---F (F = merge commit, has TWO parents: E and D)
\ /
feature: C---D---/

REBASE result (REWRITES feature's commits to sit AFTER E - looks like it was always linear):
main: A---B---E
\
feature: C'---D' (C' and D' are NEW commits - same changes, different commit IDs!)
```

3. Code Example

```
git checkout feature/login
git rebase main # replays feature/login's commits ON TOP OF main's latest commit
```

4. The Golden Rule of Rebasing — NEVER Rebase Shared/Public History

Never rebase commits that have ALREADY been pushed and that OTHER people might have already pulled/built upon. Since rebase REWRITES commit history (creating brand-new commit IDs), anyone who already has the OLD commits will end up with a confusing, diverged, conflict-prone history when they next pull. Rebasing is SAFE for your OWN local, not-yet-shared/not-yet-pushed work; it's DANGEROUS for shared branches.

5. Interactive Rebase — Cleaning Up Commit History Before Sharing

```
git rebase -i HEAD~3 # interactively edit the LAST 3 commits before pushing

pick a1b2c3d Add login form
squash e4f5g6h Fix typo in login form <- combines this INTO the previous commit
pick h7i8j9k Add password validation
```

This lets you SQUASH multiple small, messy "WIP"/"fix typo" commits into ONE clean, meaningful commit BEFORE sharing your work — producing a much more readable project history for reviewers.

6. Real Life Analogy

- **Restaurant:** Rebasing is like a chef re-writing their ENTIRE cooking log to make it LOOK like they started their dish AFTER the kitchen's latest inventory update, rather than showing the messy, ACTUAL chronological reality of when things really happened — useful for a CLEAN, presentable record, but dangerous if OTHER chefs have ALREADY been referencing the OLD version of that log.

7. Frequently Asked Interview Questions

- 1 **What's the key difference between merge and rebase?** Merge PRESERVES the actual branching history via a new merge commit; rebase REWRITES commits to produce a clean, linear history, as if the work happened sequentially.
 - 2 **Why is rebasing shared/pushed commits dangerous?** Rebase creates BRAND-NEW commit IDs for the rewritten commits — anyone who already pulled the OLD commits ends up with a diverged, confusing history requiring manual reconciliation.
 - 3 **What is interactive rebase used for?** Cleaning up commit history (squashing, reordering, editing commit messages) BEFORE sharing/pushing work, producing a more readable history for reviewers.
-

TOPIC 5: CHERRY-PICK

1. Introduction

What is it? `git cherry-pick` applies ONE SPECIFIC commit from ANOTHER branch onto your CURRENT branch — useful when you need JUST one particular fix/change, without merging an entire branch's full history.

2. Code Example

```
git checkout main
git cherry-pick a1b2c3d # applies ONLY that specific commit's changes onto main
```

3. Real Life Analogy

- **Restaurant:** Cherry-picking is like taking ONE SPECIFIC improved step from an experimental recipe (a branch) and applying it DIRECTLY to the current official recipe — WITHOUT adopting all the OTHER, still-experimental changes from that same recipe attempt.

4. A Classic Real-World Use Case — Hotfixes

Scenario: A critical bug fix was committed on a "develop" branch, but production ("main") needs it IMMEDIATELY, without pulling in ALL the other in-progress, not-yet-ready features also sitting on "develop".

```
git checkout main
git cherry-pick <the-specific-bugfix-commit-hash>
git push origin main # production gets JUST the fix, nothing else
```

5. Edge Cases

Case	Result
The cherry-picked commit's changes CONFLICT with the current branch's code	A merge conflict occurs, requiring manual resolution (Topic 9), exactly like a regular merge conflict
Cherry-picking the SAME commit TWICE onto the same branch	Usually succeeds (applying the same change again), though Git may warn if it detects the change is already present

6. Frequently Asked Interview Questions

- 1 **What does `git cherry-pick` do?** Applies ONE specific commit's changes from another branch onto your current branch, without merging that branch's entire history.
- 2 **Give a real-world scenario where `cherry-pick` is the right tool.** Urgently applying a single critical bug fix from a `develop` branch directly to `main/production`, without pulling in other still-in-progress features also present on `develop`.

TOPIC 6: STASH

1. Introduction

What is it? `git stash` TEMPORARILY SAVES your uncommitted changes (both staged and unstaged) SEPARATELY, reverting your working directory to a clean state — letting you switch tasks/branches QUICKLY without committing incomplete work.

2. Code Example

```
git stash # saves ALL current uncommitted changes, cleans your working directory
git stash list # view all stashed changes (you can have MULTIPLE stashes)
git stash pop # re-applies the MOST RECENT stash AND removes it from the stash list
git stash apply # re-applies the most recent stash but KEEPS it in the stash list too
git stash drop # deletes a stash without applying it
git stash save "WIP: login form" # stash with a descriptive message, for clarity when you have MULTIPLE stashes
```

3. A Classic Real-World Scenario

Scenario: You're mid-way through a feature on your branch when your manager says "URGENT: fix this production bug RIGHT NOW" - but your current work isn't ready to commit yet.

```
git stash # save your in-progress work, clean working directory
git checkout main
git checkout -b hotfix/urgent-bug
# ... fix the urgent bug, commit, push, get it deployed ...
git checkout feature/my-original-work
git stash pop # resume EXACTLY where you left off
```

4. Real Life Analogy

- **Restaurant:** `git stash` is like a chef quickly setting aside their HALF-PREPARED dish on a side tray (without throwing it away or serving it half-done) to immediately handle an urgent, unrelated request — then picking their half-finished dish back up EXACTLY where they left off once the urgent matter is handled.

5. Frequently Asked Interview Questions

- 1 **What does `git stash` do?** Temporarily saves uncommitted changes separately and reverts the working directory to a clean state, without requiring an actual commit.
- 2 **What's the difference between `git stash pop` and `git stash apply`?** `pop` re-applies the stash AND removes it from the stash list; `apply` re-applies it but KEEPS it in the list too (useful for applying the same stash to multiple branches).
- 3 **Give a real-world scenario for using `git stash`.** Needing to quickly switch to fix an urgent production bug while in the middle of unfinished, not-yet-committable feature work.

TOPIC 7: TAGS

1. Introduction

What is it? A tag is a PERMANENT, NAMED pointer to a SPECIFIC commit — typically used to mark RELEASE points (`v1.0.0`, `v2.3.1`) in a project's history. Unlike branches, tags DON'T move as new commits are added.

2. Code Example

```
git tag v1.0.0 # LIGHTWEIGHT tag - just a simple pointer, no extra metadata
git tag -a v1.0.0 -m "First stable release" # ANNOTATED tag - includes a message, author, date (RECOMMENDED)
git tag # list all tags
git push origin v1.0.0 # tags are NOT pushed automatically - must push explicitly
git push origin --tags # push ALL tags at once
git checkout v1.0.0 # inspect the codebase EXACTLY as it was at that release
```

3. Lightweight vs Annotated Tags

Aspect	Lightweight Tag	Annotated Tag (RECOMMENDED for releases)
Metadata	None — just a pointer	Includes tagger name, date, and a message
Use case	Quick, private, local bookmarks	Official release markers, shared with the team

4. Real Life Analogy

- **Restaurant:** A tag is like a restaurant PERMANENTLY bookmarking "this is EXACTLY our official Summer 2024 Menu" — a fixed, named reference to a specific historical point, unlike the constantly-evolving "current menu" (a branch), which keeps changing over time.

5. Frequently Asked Interview Questions

- 1 **What is a Git tag used for?** Marking a specific, permanent point in history (typically a release version) with a fixed, named reference.
 - 2 **What's the difference between a lightweight and an annotated tag?** An annotated tag includes metadata (tagger, date, message); a lightweight tag is just a simple pointer with no extra information — annotated tags are recommended for actual releases.
 - 3 **Are tags pushed to a remote automatically with a regular `git push`?** No — tags must be pushed explicitly (`git push origin <tag>` or `git push origin --tags`).
-
-

TOPIC 8: PULL REQUESTS

1. Introduction

What is it? A Pull Request (PR) — called a "Merge Request" on GitLab — is a GitHub/GitLab/Bitbucket PLATFORM feature (NOT a raw Git command) that proposes merging one branch into another, providing a structured space for CODE REVIEW, automated CI checks, and discussion BEFORE the merge actually happens.

2. The Standard PR Workflow

1. Create a feature branch: `git checkout -b feature/add-search`
2. Make commits on it: `git commit -m "Add search endpoint"`
3. Push it to the remote: `git push origin feature/add-search`
4. Open a Pull Request on GitHub/GitLab (via the web UI), targeting 'main'
5. AUTOMATED CI checks run (tests, linting, build) - covered in the CI/CD section later
6. TEAMMATES review the code, leave comments, request changes
7. You push MORE commits addressing feedback (the SAME PR updates automatically)
8. Once approved AND checks pass, the PR is MERGED into 'main' (often via the platform's UI)
9. The feature branch is typically DELETED after merging

3. Why PRs Matter for Team Collaboration (Critical for Real Jobs)

- **Code review gate:** NOTHING reaches `main` without at least one other human (and automated checks) reviewing it first — catches bugs, enforces consistency, and spreads knowledge across the team.
- **Discussion history:** PR comments create a PERMANENT record of WHY a change was made a certain way — invaluable when revisiting old code months/years later.
- **CI/CD integration:** PRs are typically the TRIGGER POINT for automated test suites, and "branch protection rules" (covered in Topic 10) can REQUIRE these checks to pass before merging is even allowed.

4. Real Life Analogy

- **Restaurant:** A Pull Request is like a chef proposing to add their experimental dish to the OFFICIAL menu — before it's actually added, the head chef (a reviewer) tastes it, gives feedback, and only once it's genuinely

APPROVED does it get folded into what customers actually receive — nothing reaches the real menu without this deliberate review gate.

5. Frequently Asked Interview Questions

- 1 **Is "Pull Request" a raw Git command?** No — it's a feature provided by hosting platforms (GitHub, GitLab), not part of Git itself; the underlying Git operation is still just a merge.
- 2 **What's the primary purpose of a Pull Request in a team workflow?** Providing a structured space for code review and automated CI checks BEFORE code is merged into a shared branch like `main`.
- 3 **What typically happens automatically when a PR is opened?** CI pipelines run automated checks (tests, linting, builds) against the proposed changes.

TOPIC 9: CONFLICT RESOLUTION

1. Introduction

What is it? A merge conflict occurs when Git CANNOT automatically combine changes — typically because TWO branches modified the SAME lines of the SAME file differently — requiring a HUMAN to manually decide the correct final result.

2. Full Walkthrough — Resolving a Real Conflict

```
git checkout main
git merge feature/pricing-update

Auto-merging pricing.txt
CONFLICT (content): Merge conflict in pricing.txt
Automatic merge failed; fix conflicts and then commit the result.
```

The conflicted file now looks like this:

```
<<<<<< HEAD
price = 100
=====
price = 120
>>>>>> feature/pricing-update
```

- `<<<<<< HEAD` through `=====`: YOUR current branch's version (`main`).
- `=====` through `>>>>>> feature/pricing-update`: the INCOMING branch's version.

Manually resolve it (edit the file to keep the CORRECT final result, removing ALL the conflict markers):

```
price = 120
```

Then complete the merge:

```
git add pricing.txt # marks the conflict as RESOLVED
git commit # completes the merge with a merge commit
```

3. Real Life Analogy

- **Restaurant:** A merge conflict is like TWO chefs independently changing the SAME dish's official price on the SAME day — the system flags "these two prices DISAGREE, a HUMAN must decide the final correct number," rather than the system silently guessing/picking one arbitrarily.

4. Tools That Help

```
git mergetool # opens a VISUAL diff/merge tool (if configured) instead of manually editing conflict markers
git merge --abort # completely CANCELS an in-progress, unresolved merge, reverting to the pre-merge state
```

5. Edge Cases

Case	Result
Aborting a merge mid-conflict-resolution	<code>git merge --abort</code> cleanly reverts everything — as if the merge attempt never happened
Forgetting to remove ALL conflict markers before committing	The committed file literally contains the broken <code><<<<<<</>>>>>> text</code> — a real, embarrassing, surprisingly common mistake

6. Common Mistakes

- Forgetting to remove the conflict marker lines themselves before committing, accidentally committing broken code containing literal `<<<<<<<` text.
- Panicking and force-pushing over conflicts instead of resolving them properly — can destroy teammates' work.

7. Frequently Asked Interview Questions

- 1 **What causes a merge conflict?** Two branches modifying the same lines of the same file in incompatible ways, leaving Git unable to automatically determine the correct combined result.
- 2 **What do the `<<<<<<<`, `=====, and >>>>>>> markers mean in a conflicted file?`** They delineate YOUR current branch's version (above `=====`) from the INCOMING branch's version (below `=====`) for that conflicting section.
- 3 **How do you completely cancel an in-progress merge with unresolved conflicts?** `git merge --abort`.
- 4 **What are the exact steps to resolve a conflict?** Manually edit the file to the correct final content (removing ALL conflict markers), `git add` the resolved file, then `git commit` to complete the merge.

TOPIC 10: REAL-WORLD TEAM WORKFLOWS & SCENARIO-BASED COMMAND REFERENCE

1. Introduction

This topic directly addresses how LARGE teams, with MANY developers working on the SAME codebase (often the SAME files), avoid stepping on each other's work and keep `main` always correct and deployable — the practical, production-level knowledge beyond individual Git commands.

2. How Large Teams Avoid Conflicts on Shared Files — The Core Strategies

Strategy 1: Small, Short-Lived Feature Branches

Each developer works on a SMALL, focused branch (e.g., "add email validation") for HOURS or a FEW DAYS at most - NOT weeks. The SHORTER a branch lives, the LESS chance it has to diverge significantly from `main` and cause painful conflicts later.

Strategy 2: Frequent Integration (Pulling `main` Often)

```
git checkout feature/my-work
git pull origin main --rebase # regularly incorporate main's latest changes INTO your branch
# (or 'git merge main' - teams choose ONE consistent approach)
```

Doing this DAILY (or even more often) means you discover/resolve SMALL conflicts EARLY and INCREMENTALLY, rather than facing one MASSIVE, painful conflict after weeks of divergence.

Strategy 3: Branch Protection Rules (Enforced by GitHub/GitLab, Not Git Itself)

- REQUIRE at least 1-2 approving code reviews before merging into `main`
- REQUIRE all CI checks (tests, linting, build) to PASS before merging is even allowed
- DISALLOW direct pushes to `main` entirely - EVERYONE must go through a Pull Request
- REQUIRE branches to be up-to-date with `main` before merging (forces regular rebasing/merging)

Strategy 4: Modular Code Architecture (Reduces WHO Touches the SAME Files)

Well-designed codebases (proper package/module boundaries, clear separation of concerns - exactly the MVC/layered architecture principles from Advanced Java) mean different teams/developers NATURALLY work in DIFFERENT files/modules most of the time, reducing the FREQUENCY of true conflicts simply by good design.

Strategy 5: Trunk-Based Development vs Git Flow — Two Common Team Branching Models

Model	How It Works	Best For
Trunk-Based Development	Everyone commits SMALL, frequent changes DIRECTLY to (or via very short-lived branches merged quickly into) <code>main/trunk</code> ; feature flags hide incomplete work in production	Modern, fast-moving teams with strong CI/CD and automated testing
Git Flow	Separate long-lived <code>develop</code> , <code>feature/*</code> , <code>release/*</code> , and <code>hotfix/*</code> branches, with a more formal, staged release process	Larger, more traditional release cycles (e.g., scheduled quarterly releases)
GitHub Flow	A simpler middle ground: <code>main</code> is ALWAYS deployable, feature branches merge in via PR after review/CI, deployed immediately after merge	Most modern SaaS/web teams — a common, pragmatic default

Strategy 6: Feature Flags (Decoupling "merged" from "released")

```
if (featureFlagService.isEnabled("new-checkout-flow", currentUser)) {  
  return newCheckoutFlow();  
} else {  
  return oldCheckoutFlow();  
}
```

This lets teams MERGE incomplete/in-progress features into `main` FREQUENTLY (avoiding long-lived, conflict-prone branches) while keeping that functionality HIDDEN from real users until it's genuinely ready — a cornerstone technique of trunk-based development at scale (used at companies like Google, Facebook, Netflix).

Strategy 7: Code Owners (Automatically Requesting the RIGHT Reviewers)

```
# CODEOWNERS file (GitHub feature)  
/src/payments/ @payments-team  
/src/auth/ @security-team
```

Automatically requests review from the RIGHT specialized team whenever a PR touches files in their designated area — ensuring the people with the MOST context on sensitive code (like payments/auth) always review changes to it.

3. Real Life Analogy

- **Restaurant Chain:** A large restaurant chain avoids chaos across HUNDREDS of chefs by: (1) having each chef work on SMALL, quick experimental tweaks rather than months-long secret recipe overhauls, (2) constantly syncing with the CURRENT official menu rather than working in isolation for a long time, (3) requiring a HEAD CHEF's sign-off before anything reaches the real menu, (4) organizing chefs into SPECIALIZED stations (so the dessert chef and the grill chef naturally aren't touching the SAME dish simultaneously), and (5) sometimes secretly testing a new dish with a SMALL subset of regular customers (a feature flag) before rolling it out to everyone.

4. SCENARIO-BASED GIT COMMAND REFERENCE (Interview & Real-World Practice)

Scenario	Command(s) to Use
I want to start tracking a brand-new project with Git	<code>git init</code>
I want to get a copy of an existing remote project	<code>git clone <url></code>
I made changes but haven't saved them to history yet	<code>git add <files></code> then <code>git commit -m "message"</code>
I want to see what's changed but not yet committed	<code>git status</code> and <code>git diff</code>
I want to create a new feature branch and start working on it immediately	<code>git checkout -b feature/my-feature</code>
I need to switch away from my current branch WITHOUT committing half-done work	<code>git stash</code> , then switch branches, later <code>git stash pop</code>
I need to urgently fix a production bug while mid-feature	<code>git stash</code> → <code>git checkout main</code> → <code>git checkout -b hotfix/xyz</code> → fix → PR → merge → <code>git checkout back</code> → <code>git stash pop</code>
I want to bring <code>main</code> 's latest changes into my feature branch (clean, linear history preferred)	<code>git rebase main</code> (while on the feature branch)
I want to bring <code>main</code> 's latest changes into my feature branch (preserve exact history)	<code>git merge main</code> (while on the feature branch)
I need JUST one specific bug-fix commit from another branch, not the whole branch	<code>git cherry-pick <commit-hash></code>
My last commit message has a typo, and I haven't pushed yet	<code>git commit --amend</code>
I committed to the wrong branch by mistake	<code>git reset HEAD~1</code> (undo the commit, keep changes) → switch to the CORRECT branch → commit there
I want to completely discard all my uncommitted local changes	<code>git checkout -- <file></code> (single file) or <code>git reset --hard</code> (everything, DANGEROUS)
I want to see exactly who changed a specific line, and when	<code>git blame <file></code>
I want to find WHICH commit introduced a specific bug, across many commits	<code>git bisect start</code> (binary-searches through history automatically)
I hit a merge conflict and want to just start over	<code>git merge --abort</code> (during a merge) or <code>git rebase --abort</code> (during a rebase)
I want to mark this exact commit as an official release	<code>git tag -a v1.2.0 -m "Release 1.2.0"</code> then <code>git push origin v1.2.0</code>
I want to permanently delete a branch that's already merged	<code>git branch -d feature/old-feature</code> (locally), <code>git push origin --delete feature/old-feature</code> (remote)
I accidentally pushed a commit with a secret/password in it	Rewrite history to REMOVE it (<code>git filter-repo</code> or BFG Repo-Cleaner) AND immediately rotate/invalidate that secret — deleting the commit alone is NOT sufficient once pushed
I want to see the full history of commits in a readable, graph form	<code>git log --oneline --graph --all</code>
My teammate's PR has commits I want to test locally before approving	<code>git fetch origin</code> then <code>git checkout <their-branch-name></code>
I need to update my LOCAL branch to match a rewritten (force-pushed) remote branch	<code>git fetch origin</code> then <code>git reset --hard origin/<branch-name></code>

Scenario	Command(s) to Use
Two people are editing the SAME file, and I want to avoid a painful conflict later	Rebase/merge <code>main</code> into your branch FREQUENTLY (daily), keep the branch SHORT-LIVED, communicate with the teammate about which sections each of you is touching

5. Common Mistakes

- Letting feature branches live for WEEKS without syncing with `main`, guaranteeing a large, painful conflict eventually.
- Pushing directly to `main` in a team setting without going through PR review/CI checks — branch protection rules exist specifically to prevent this.
- Force-pushing (`git push --force`) to a SHARED branch other teammates have already pulled — rewrites history out from under them, causing serious confusion; use `--force-with-lease` (safer, checks nobody else has pushed since your last fetch) if force-pushing is genuinely necessary on a personal branch.

6. Best Practices

- Keep feature branches SMALL and SHORT-LIVED; integrate with `main` frequently.
- Enforce branch protection rules (required reviews, required passing CI) on `main` in any real team project.
- Use feature flags to decouple "merging code" from "releasing functionality to users," enabling trunk-based development even for large, in-progress features.
- Communicate proactively with teammates when you KNOW you're both about to touch the same file/area.

7. Frequently Asked Interview Questions

- 1 How do large teams avoid constant merge conflicts when many developers work on the same codebase?** Small, short-lived branches; frequent integration with `main`; branch protection rules requiring review/CI before merge; modular architecture reducing file overlap; and techniques like feature flags and trunk-based development.
- 2 What is trunk-based development, and why do companies like Google/Facebook use it at scale?** A model where developers commit small, frequent changes directly to (or via very short-lived branches into) `main`, using feature flags to hide incomplete work — it minimizes long-lived branch divergence and the resulting painful conflicts, while still allowing continuous integration and deployment.
- 3 What is a feature flag, and how does it help team collaboration?** A runtime toggle that hides incomplete functionality from real users while still allowing the underlying code to be merged into `main` early and frequently, decoupling "code is merged" from "feature is live."
- 4 What do "branch protection rules" typically enforce?** Required code review approvals, required passing CI checks, and disallowing direct pushes to `main` — all enforced by the hosting platform (GitHub/GitLab), not Git itself.

- 5 **Why is `git push --force` on a SHARED branch dangerous, and what's the safer alternative?** It rewrites history that teammates may have already pulled, causing confusing divergence; `git push --force-with-lease` is safer, refusing to overwrite the remote if someone else has pushed since your last fetch.

8. Revision Notes (Entire Git Section Consolidated)

GIT — CHEAT SHEET

=====

DISTRIBUTED VCS: every clone has the FULL history

THREE AREAS: Working Directory -> (`git add`) -> Staging Area -> (`git commit`) -> Repository

BRANCHING: lightweight pointers to commits, NOT full copies

MERGE: fast-forward (no divergence) vs three-way (creates a merge commit)

REBASE: rewrites commits for a LINEAR history - NEVER rebase already-PUSHED/shared commits!

CHERRY-PICK: apply ONE specific commit from elsewhere (classic hotfix use case)

STASH: temporarily shelve uncommitted work to switch context quickly

TAGS: permanent, non-moving pointers marking releases (annotated > lightweight for real releases)

PULL REQUESTS: a PLATFORM feature (not raw Git) - code review + CI gate before merging

CONFLICT RESOLUTION: <<<<<< HEAD / ===== / >>>>>> markers -> manually fix -> `git add` -> `git commit`
`git merge --abort` / `git rebase --abort` to bail out entirely

TEAM SCALE STRATEGIES: small short-lived branches, frequent main integration, branch protection rules (required review + CI), modular architecture, feature flags, trunk-based dev / GitHub Flow / Git Flow, CODEOWNERS for automatic reviewer routing

9. Homework (Covers Entire Git Section)

- 1 Create a repo, make a feature branch, deliberately cause a merge conflict with `main`, and resolve it fully by hand.
- 2 Practice the full stash workflow: start a feature, stash it, create and merge a hotfix branch, then pop your stash back.
- 3 Use interactive rebase to squash 3 messy commits into 1 clean commit before an imagined "push."
- 4 Write out, in your own words, which branching model (trunk-based, Git Flow, GitHub Flow) you'd recommend for a 5-person fast-moving startup vs. a 200-person team shipping quarterly enterprise releases, and why.

FINAL SUMMARY — GIT COMPLETE

This concludes the **Git** section — 10 topics (9 core + 1 bonus):

- 1 **Repository** — the three areas, distributed vs centralized VCS.
- 2 **Branching** — lightweight commit pointers.
- 3 **Merge** — fast-forward vs three-way.
- 4 **Rebase** — linear history, and the golden "never rebase shared history" rule.
- 5 **Cherry-pick** — applying single commits, the classic hotfix scenario.

- 6 **Stash** — temporarily shelving work, the "urgent bug mid-feature" scenario.
- 7 **Tags** — permanent release markers.
- 8 **Pull Requests** — the code-review/CI gate (a platform feature, not raw Git).
- 9 **Conflict Resolution** — a full manual walkthrough with real conflict markers.
- 10 **Real-World Team Workflows & Scenario Reference** — how large teams avoid conflicts at scale (small branches, frequent integration, branch protection, feature flags, trunk-based development), plus a full scenario-to-command lookup table.

You now have both the individual Git commands AND the practical, team-scale collaboration knowledge that separates "knows Git commands" from "can work effectively on a real production team." Next: Docker, covering how this code actually gets packaged and deployed.

(END OF GIT MASTER NOTES — ready to proceed to Docker whenever you say "Next Topic.")